HYDRO Technical Documentation

Water Safety and Maintenance Reporting System

Item	Value
Project	HYDRO
Scope	Current repository implementation
Platform	Firebase Hosting, Cloud Functions, Firestore, Storage, Authentication
Build Model	Static frontend build to `dist/` with serverless backend functions
Date	April 2026

Executive Summary

HYDRO is a role-based web application for reporting, tracking, and resolving campus water-related issues. The current system provides separate interfaces for students, maintenance staff, administrators, and super administrators while sharing a single Firebase backend.

The core purpose of the system is to:

- collect water issue reports with structured location and issue data
- route reports into an administrative review and assignment process
- track maintenance work from assignment through completion
- publish role-targeted announcements
- manage accounts, locations, analytics, and audit visibility in a controlled backend

At runtime, the frontend is served from Firebase Hosting, the browser uses the Firebase Web SDK for authentication and approved data access, and Cloud Functions handle privileged operations such as user provisioning, report assignment, announcements, bootstrap, and audited updates.

Technology Stack

Layer	Current implementation
Frontend	Static HTML, CSS, and ES module JavaScript in `public/`
Frontend SDK	Firebase Web SDK `10.8.0` loaded from `www.gstatic.com`
Backend	Firebase Cloud Functions v2 in `functions/`
Functions runtime	Node.js 20
Database	Cloud Firestore
File storage	Firebase Storage
Authentication	Firebase Authentication with email/password and admin TOTP MFA pages
Hosting	Firebase Hosting serving `dist/`
Build tools	`scripts/build.mjs` and `esbuild`
UI assets	Local Font Awesome assets and QR generator vendor files in `public/vendor/`

System Architecture

High-level architecture

HYDRO uses a Firebase-centered architecture with a static client and a serverless backend:

1. Users load HTML pages from Firebase Hosting.
2. Frontend modules initialize Firebase Authentication, Firestore, Storage, and App Check.
3. The browser reads and writes directly to Firestore and Storage only where security rules allow it.
4. Privileged operations run through callable Cloud Functions.
5. Firestore triggers and scheduled functions handle audit and maintenance automation.

Runtime interaction model

Layer	Responsibility
`public/*.html` and `public/js/**`	UI rendering, auth bootstrap, role routing, and user interactions
Firebase Authentication	Sign-in, password reset, invited-account setup, admin multi-factor sign-in
Cloud Firestore	Reports, tasks, users, announcements, locations, audit data, and configuration
Firebase Storage	Report images, avatars, and announcement attachments
Cloud Functions	Assignment, profile updates, user invites, announcements, bootstrap, report media linking, analytics, and audit support
Firestore / Storage Rules	Role-based and ownership-based data access enforcement

Direct versus privileged operations

- Direct client operations:
- sign-in and password reset through Firebase Auth
- report creation under Firestore rules
- report image upload to Storage under Storage rules
- allowed reads for reports, tasks, announcements, users, locations, and audit logs based on role
- Callable function operations:
- user invitation, archival, and role changes
- report status updates and task assignment
- maintenance task lifecycle transitions
- profile updates
- announcement creation and updates

- bootstrap and account activation
- login attempt tracking and unread announcement state

Codebase Structure

Root layout

Path	Purpose
`README.md`	Portfolio-facing project overview
`docs/`	Documentation set
`public/`	Frontend source
`functions/`	Cloud Functions source
`dist/`	Hosting build output
`scripts/`	Build and maintenance scripts
`firebase.json`	Hosting, Functions, emulator, and header configuration
`.firebaserc`	Firebase project alias and hosting target mapping
`firestore.rules`	Firestore security rules
`firestore.indexes.json`	Firestore composite index definitions
`storage.rules`	Storage security rules

Frontend structure: `public/`

Current frontend routes and assets are organized as follows:

- student and shared pages: `index.html`, `MyRecords.html`, `notifications.html`, `profile.html`, `login.html`, `auth-action.html`, `bootstrap.html`, `signup.html`, `404.html`
- admin pages: `public/admin/`
- maintenance pages: `public/maintenance/`
- frontend modules: `public/js/`
- shared styles: `public/css/`
- static assets: `public/assets/`
- vendored libraries: `public/vendor/`

Key frontend module groups:

Path	Responsibility
`public/js/auth/`	Auth bootstrap, active-account checks, role guards, and action handling
`public/js/reports/`	Report schema, options, and Firestore service layer
`public/js/ui/`	Submit modal, lightbox, navigation, and mobile layout helpers
`public/js/config/`	Shared constants for routes, roles, collections, and statuses
`public/js/shared/`	Logging, rate-limit messaging, device, and navigation helpers

Backend structure: `functions/`

Path	Responsibility
`functions/index.js`	Function export registry
`functions/admin/`	Admin callables for reports, users, announcements, analytics, and locations
`functions/maintenance/`	Maintenance task and dashboard callables
`functions/bootstrap/`	First super-admin bootstrap callables
`functions/shared/`	Shared auth, audit, account, profile, rate-limit, media, and config modules
`functions/triggers/`	Firestore-triggered audit handlers
`functions/scripts/`	Utility scripts

Build output: `dist/`

`dist/` is the Hosting deployment target. It is rebuilt from `public/` by `scripts/build.mjs` and should be treated as output rather than source.

User Roles

The current role model is implemented in both frontend constants and backend auth helpers.

Role	Current responsibilities
`student`	Submit reports, view own report history, read announcements, manage own profile
`maintenance`	View assigned tasks, accept and start work, complete tasks, view maintenance dashboard data, manage own settings

Role	Current responsibilities
`admin`	Review reports, assign tasks, manage announcements, users, locations, analytics, and audit visibility
`super_admin`	All admin capabilities plus elevated role-management and first-run bootstrap authority

Supporting account states used in the current codebase:

State field	Values
`role`	`student`, `maintenance`, `admin`, `super_admin`
`status`	`invited`, `active`, `disabled`
`activated`	boolean active flag used with invited-account setup
`isArchived`	archive flag that blocks active access

Core Workflows

Authentication flow

The sign-in flow is centered in `public/js/auth.js` and `public/js/auth/auth-bootstrap.js`.

1. The user submits email and password from `login.html`.
2. The client calls `checkLoginAllowed` before credential submission to enforce login-attempt lockout.
3. Firebase Authentication processes `signInWithEmailAndPassword`.
4. Successful first-factor sign-in records a successful login attempt and then checks `users/{uid}` for active-account status.
5. The app blocks archived, disabled, invited, or missing profiles.
6. The user is routed to the correct dashboard based on role.
7. If Firebase returns `auth/multi-factor-auth-required`, the app stores pending sign-in state and redirects to `admin/mfa-challenge.html`.

Related supporting flows:

- password reset emails are sent from `login.html` using `buildActionCodeSettings()`
- invited-account setup is completed through `auth-action.html` and `activateInvitedAccount`
- admin MFA enrollment and challenge pages use TOTP through the Firebase Auth SDK

Report submission flow

The primary report-submission UI is the shared submit modal used on the student-facing pages.

1. The frontend loads buildings, floors, issue types, and risk levels through `getSystemConfig`.
2. `createStudentReport` validates the payload against the frontend schema and current system options.
3. The client creates a Firestore `reports` document with status `pending`.
4. Firestore rules require server timestamps, a valid location payload, canonical issue type values, and `riskLevel` defaulting to `low` on create.
5. Optional report images are uploaded to Storage under `reports/{reportId}/{fileName}`.
6. The client calls `appendReportPhotos` to attach uploaded image URLs to the report document.
7. Firestore triggers record audit entries for report creation and image attachment.

Admin review and assignment flow

Admin review is implemented through `public/admin/reports.html`, `public/admin/report-details.html`, and `functions/admin/index.js`.

1. Admins load paginated report data through `getAllReports`.
2. Admins can update report status, notes, and risk level through `updateReportStatus`.
3. Assignment uses `assignTask`, which validates the report state and assignee list.
4. One task document is created per assignee.
5. The report is updated to `in_progress` and stores both legacy single-assignee fields and the current multi-assignee fields.
6. Assignment writes an audit entry and computes the task priority from the report risk level.

Maintenance task lifecycle flow

Maintenance workflow is implemented in `functions/maintenance/index.js` and the maintenance pages under `public/maintenance/`.

Task lifecycle states used in the current code:

Status	Meaning
`assigned`	Task has been assigned and is waiting for acceptance
`accepted`	Assignee accepted the task
`in_progress`	Work has started
`completed`	Work finished successfully

Status	Meaning
`expired`	Acceptance deadline passed before acceptance
`closed_by_team`	Another assignee completed the same team job

Workflow:

1. Maintenance users load assigned tasks through `getMyTasks`.
2. `acceptTask` confirms assignment ownership and rejects overdue tasks.
3. `startTask` transitions an accepted task into `in_progress`.
4. `markTaskDone` requires completion notes, records timestamps and duration, resolves the linked report, and closes remaining non-terminal sibling tasks for the same report.
5. `expireOverdueTasks` runs on a schedule every 15 minutes to mark overdue unaccepted tasks as `expired` and clean stale ephemeral documents.

Announcement flow

Announcements are part of the active operational workflow.

1. Admins create, update, and delete announcements through admin callables.
2. Announcement documents store title, body, type, audience, active state, and pinned state.
3. Frontend announcement loaders query only active announcements visible to the current role.
4. Unread state is tracked with `getUnreadAnnouncementsCount` and `markAnnouncementsSeen` using `announcementsLastSeenAt` on the user document.

FAQ note

The repository still contains FAQ-related frontend and backend artifacts, but they are not part of the active documented operational feature set for this public release because they are not fully integrated into the current administrative workflow.

Bootstrap flow

The repository includes a first-run bootstrap path for creating the first super admin.

1. An operator prepares `systemConfig/bootstrap` with `bootstrapEnabled` and `bootstrapSecretHash`.
2. An authenticated user opens `bootstrap.html`.
3. The client checks `getBootstrapStatus`.
4. The user submits the bootstrap key to `bootstrapFirstSuperAdmin`.
5. The function validates the key hash, confirms that no super admin already exists, creates the `users/{uid}` record as `super_admin`, and disables bootstrap.

Backend Functions Overview

All deployed functions are exported from `functions/index.js`.

Shared, account, and system functions

Function	Purpose
`updateUserProfile`	Audited profile updates for name, phone, and avatar data
`appendReportPhotos`	Attach uploaded image URLs to a report
`syncReportPhotosFromStorage`	Rebuild report image URLs from Storage for admin repair use
`getSystemConfig`	Return location and reporting configuration used by the frontend
`bootstrapFirstSuperAdmin`	Provision the first super admin from an armed bootstrap state
`getBootstrapStatus`	Return bootstrap readiness state
`activateInvitedAccount`	Transition invited users into active accounts
`logUnprovisionedLogin`	Write an audit marker when Auth exists but `users/{uid}` is missing
`checkLoginAllowed`	Pre-auth lockout check for repeated failures
`recordLoginAttempt`	Track credential success and failure attempts
`manageDepartments`	Department maintenance callable
`getDepartments`	Return active department data

Admin functions

Function	Purpose
`getAllReports`	Paginated report management query
`updateReportStatus`	Update report status, notes, and risk level
`assignTask`	Create maintenance tasks and move the report into active work
`getMaintenanceStaff`	Return maintenance staff and active workload counts
`createAnnouncement`	Create role-targeted announcements
`updateAnnouncement`	Update announcement fields

Function	Purpose
`deleteAnnouncementByAdmin`	Soft-delete announcements
`getAnalytics`	Return administrative analytics data
`searchUsers`	Search users by prefix
`getAllUsers`	Paginated user list with filtering
`updateUserRole`	Update role, department, and status fields
`createUserByAdmin`	Create invited user accounts
`resendInviteByAdmin`	Resend setup metadata for invited users
`setUserArchiveStatus`	Archive or restore accounts
`getLocations`	Return active locations
`createLocation`	Create building records
`deleteLocation`	Delete building records
`createFloor`	Create floor records under a building
`deleteFloor`	Delete floor records
`logAuditDashboardViewed`	Record audit dashboard access
`getUnreadAnnouncementsCount`	Return unread announcement counts
`markAnnouncementsSeen`	Update announcement last-seen state

Maintenance functions

Function	Purpose
`getMyTasks`	Return the current maintenance user's tasks
`getMyTaskBadgeCounts`	Return awaiting, urgent, and expired task counts
`acceptTask`	Accept an assigned task
`startTask`	Start an accepted task
`markTaskDone`	Complete a task, resolve the report, and close sibling tasks
`getMyStats`	Return maintenance dashboard totals

Function	Purpose
`expireOverdueTasks`	Scheduled task expiration and cleanup job

Triggers

Function	Trigger
`onReportCreatedAudit`	Firestore create trigger for reports
`onReportImagesAddedAudit`	Firestore update trigger for report image changes

Database Structure

The following collections are part of the active operational model in the current repository.

Core operational collections

Collection	Purpose	Key fields used by current code
`users`	Role and account state	`email`, `name`, `role`, `phone`, `department`, `avatarUrl`, `status`, `activated`, `isArchived`, `announcementsLastSeenAt`, timestamps
`students`	Student profile details	`email`, `name`, `phone`, `avatarUrl`, timestamps
`reports`	Water issue reports	`studentId`, `status`, `building`, `floor`, `location`, `issueType`, `riskLevel`, `description`, `photoUrl`, `photoUrls`, `createdByName`, `reporterSnapshot`, `assignedTo`, `assignedToUids`, `taskId`, `taskIds`, timestamps
`tasks`	Assigned maintenance work	`reportId`, `assignedTo`, `assignedBy`, `status`, `priority`, `notes`, `expectedResolution`, `acceptBy`, lifecycle timestamps, `durationSeconds`, reporter fields, `reportSnapshot`, `completionNotes`
`announcements`	Role-targeted announcement feed	`title`, `body`, `type`, `audience`, `isActive`, `pinned`, `isDeleted`, creator fields, timestamps
`locations`	Building and floor configuration	`building`, `floor`, `area`, `active`, timestamps

Collection	Purpose	Key fields used by current code
`audit_logs`	Immutable audit trail	`timestamp`, `actorUid`, `actorRole`, `actorName`, `actionType`, `targetType`, `targetId`, `metadata`
`systemConfig`	Restricted system settings	bootstrap configuration document fields
`securityLogs`	Bootstrap and security-related control data	bootstrap rate-limit state

Supporting collections used by backend controls

Collection	Purpose	Notes
`departments`	Department data	Supported by exported callables
`rate_limits`	Per-user callable rate limiting	Managed by backend only
`login_attempts`	Login lockout state	Managed by backend only

Storage paths used by the current system

Path pattern	Purpose
`reports/{reportId}/{fileName}`	Current report image upload path
`reports/{studentId}/{reportId}/{fileName}`	Legacy report image path still supported by rules
`avatars/{userId}/{fileName}`	User avatar uploads
`announcements/{announcementId}/{fileName}`	Announcement attachments

Security & Access Control

Authentication handling

- Authentication uses Firebase Auth with email/password sign-in.
- Password reset and invited-account activation rely on action URLs.
- Admin multi-factor authentication uses TOTP enrollment and challenge pages.
- The frontend blocks use of accounts that are archived, disabled, invited, or missing a `users/{uid}` profile.

Role verification

Backend role checks are centralized in `functions/shared/auth.js`.

Current backend helpers:

- `ensureActiveUser`
- `ensureAdmin`
- `ensureSuperAdmin`
- `ensureMaintenance`

These helpers verify:

- the request is authenticated
- the Firestore user document exists
- `isArchived` is not `true`
- `status` is not `invited` or `disabled`
- `activated` is not `false`
- the caller role matches the function requirement

Firestore rules overview

Firestore rules enforce the following access model:

- `users`: self-read or admin-read only; client writes blocked
- `students`: self-read or admin-read only; client writes blocked
- `reports`: active users may create validated reports; creator can append photo URLs; admins can read all; maintenance can read assigned reports
- `tasks`: admin reads all; maintenance reads only assigned tasks; client writes blocked
- `announcements`: active users read only matching active announcements; admins can read all; client writes blocked
- `locations`: active users may read; client writes blocked
- `audit_logs`: admin read only; client writes blocked
- `systemConfig`, `securityLogs`: client read and write blocked

Storage rules overview

Storage rules enforce:

- active-account gating before access
- owner-only uploads for report images and avatars
- staff read access for report images
- admin-only uploads for announcement attachments

- image-only file types with a 5 MB size limit for uploads

Additional controls

- App Check is initialized in the frontend with reCAPTCHA v3 support.
- `functions/shared/rateLimit.js` applies per-user callable rate limiting with stricter policies for sensitive actions.
- `functions/shared/loginAttempts.js` enforces a five-failure, fifteen-minute lockout window keyed by email and client IP.
- Hosting applies baseline security headers and a report-only Content Security Policy.

Deployment & Configuration

Hosting and build

Item	Current configuration
Hosting directory	`dist`
Hosting site target	placeholder in `firebase.json` until configured
Build command	`npm run build`
Build behavior	copy `public/` to `dist/` and minify `dist/js/**/*.js`

Functions and runtime

Item	Current configuration
Functions source	`functions`
Runtime	Node.js 20
Primary region	`asia-southeast1`

Emulator configuration

Service	Port
Auth	`9099`
Functions	`5001`
Firestore	`8080`

Service	Port
Storage	`9199`
Hosting	`5000`
Emulator UI	`4000`

Project-specific configuration notes

When moving the repository to another Firebase project, review these files together:

- `.firebaserc`
- `firebase.json`
- `public/js/firebase-config.js`
- `public/js/auth/action-code-settings.js`
- `functions/admin/index.js`

Public template placeholders that must be configured for a real deployment:

File	Placeholder or default
`.firebaserc`	`YOUR_PROJECT_ID`
`.firebaserc` target mapping	`YOUR_HOSTING_TARGET` -> `YOUR_HOSTING_SITE`
`firebase.json`	`hosting.site = YOUR_HOSTING_SITE`
`public/js/firebase-config.js`	Firebase Web App placeholders such as `YOUR_PROJECT_ID` and `YOUR_API_KEY`
`public/js/auth/action-code-settings.js`	uses `window.location.origin`
`functions/admin/index.js`	`process.env.APP_BASE_URL `'http://localhost:5000/auth-action.html'`

`dist/` remains build output for Hosting and should be regenerated from `public/` with `npm run build` after configuration changes, not edited manually.

What To Change For A New Firebase Project

The current repository splits environment-sensitive settings across project targeting, frontend Firebase configuration, and action-link hosts.

File	What must stay aligned
`.firebaserc`	default Firebase project ID and Hosting target mapping
`firebase.json`	Hosting site name, Hosting output directory, Functions source
`public/js/firebase-config.js`	Firebase Web App values and App Check site key
`public/js/auth/action-code-settings.js`	browser-origin action URL handling for password reset and account setup
`functions/admin/index.js`	server-side action URL base for invite/setup links through `APP_BASE_URL`

For a new Firebase project, update these values together so that:

1. the frontend points at the new Firebase project
2. Hosting deploys to the new HYDRO site
3. password reset links open the new deployed `auth-action.html`
4. invited-account setup links generated by the server open the same deployed environment

APP_BASE_URL

`APP_BASE_URL` is used by Cloud Functions when generating server-side password reset and invited-account setup links.

Current usage:

- `functions/admin/index.js` builds auth links from `process.env.APP_BASE_URL || 'http://localhost:5000/auth-action.html'`

Expected values should point directly to the deployed `auth-action.html` route, for example:

- `https://your-project.web.app/auth-action.html`
- `https://your-project.firebaseapp.com/auth-action.html`
- `https://your-custom-domain.com/auth-action.html`

If `APP_BASE_URL` is missing or incorrect, server-generated auth links can point to the wrong environment.

This repository does not include a checked-in deployment config file that sets `APP_BASE_URL`, so it must be supplied through the Functions runtime environment used for deployment.

Bootstrap Setup

The first super-admin flow depends on the exact Firestore document path `systemConfig/bootstrap`.

Required fields before provisioning:

Field	Required value
`bootstrapEnabled`	`true`
`bootstrapSecretHash`	SHA-256 hash of the bootstrap key that the operator will later enter on `/bootstrap.html`

Bootstrap key validation in the current backend requires:

- 12 to 128 characters
- letters, numbers, underscore, and hyphen only
- no spaces

Provisioning sequence:

1. Create Firestore collection `systemConfig`.
2. Create document `bootstrap`.
3. Set `bootstrapEnabled = true`.
4. Set `bootstrapSecretHash` to the SHA-256 hash of the chosen bootstrap key.
5. Sign in with the Firebase Auth account that should become the first super admin.
6. Open `/bootstrap.html` on the deployed HYDRO site.
7. Enter the original raw bootstrap key.
8. Submit the form.
9. Confirm the function disables bootstrap by setting `bootstrapEnabled` to `false` and writing `lockedAt`.

Detailed operator steps are documented in `docs/setup-and-deployment.md`.

Known Limitations

The following limitations are visible in the current repository:

- The repository does not currently include automated test suites for frontend or Cloud Functions behavior.
- Admin chart rendering depends on loading `Chart.js` at runtime from a CDN on the dashboard and analytics pages.

- Department management callables are exported from the backend, but there is no dedicated page in the current `public/` routes for managing them directly.
- `dist/` remains a build output directory and can drift from source if it is edited manually instead of rebuilding from `public/`.